FLUJO DE DATOS

Flujo de Datos: Manejo de Archivos

El manejo de archivos permite a las aplicaciones interactuar con sistemas de almacenamiento. Python ofrece una interfaz sencilla para manipular archivos, tanto de texto como binarios.

Operaciones comunes

Abrir un archivo:

Con **open('nombre', 'modo')** donde el modo puede ser:

- **'r'** (lectura), **'w'** (escritura), **'a'** (añadir contenido al final), **'b'** (modo binario).
- También puedes usar combinaciones como **'rb'** para lectura binaria.

Ejemplo avanzado

Copiar archivos grandes en bloques (útil para optimizar memoria):

Copiar un archivo en bloques de 4096 bytes




```
def copiar_archivo(origen, destino):
    with open(origen, 'rb') as archivo_origen,
open(destino, 'wb') as archivo_destino:
        while bloque := archivo_origen.read(4096):  #
Lee en bloques de 4096 bytes
            archivo_destino.write(bloque)

copiar_archivo('archivo_origen.bin',
'archivo_copia.bin')
```

▸ **Nota**



Buffers: Optimización de E/S

Un **buffer** permite gestionar las operaciones de entrada y salida (E/S) de manera más eficiente. Python utiliza buffers automáticamente, pero también puedes controlar el tamaño del buffer al abrir un archivo:

Especificar el tamaño del buffer (8192 bytes en este caso)

```
with open('archivo.txt', 'w', buffering=8192) as archivo:
    archivo.write('Datos escritos usando un buffer grande.')
```

Uso práctico: Reducir accesos a disco o red, mejorando el rendimiento.

Manejo de Errores en Sockets

El manejo adecuado de errores asegura que tu aplicación sea robusta. Además de los errores básicos, puedes implementar **reintentos** en caso de fallos temporales, por ejemplo, debido a problemas de conexión.

Ejemplo: Reintentar conexión en un cliente TCP

```
import socket
import time

def conectar_con_reintentos(host, puerto, intentos=3, espera=2):
    for intento in range(intentos):
        try:
            print(f"Intentando conectar (Intento {intento + 1})...")
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cliente:
                cliente.connect((host, puerto))
                cliente.sendall(b'Hola, servidor!')
                respuesta = cliente.recv(1024)
```

```
                print(f"Respuesta del servidor: {respuesta.decode()}")
                return  # Salir si se conecta exitosamente
        except socket.error as e:
            print(f"Error de conexión: {e}")
            time.sleep(espera)  # Esperar antes de reintentar
    print("No se pudo establecer la conexión.")

conectar_con_reintentos('localhost', 12345)
```

Mejora: Utiliza bibliotecas como retry para manejar reintentos de manera más elegante.

Programación Multihilo (Threading)

En sistemas complejos, es importante gestionar tareas concurrentes, como servir múltiples clientes en una red o procesar grandes cantidades de datos. Python permite usar **multithreading** para estas tareas.

Ejercicios para la clase 10

Crear un servidor multicliente con hilos:

```
import socket
import threading

def manejar_cliente(cliente, direccion):
    print(f"Conexión desde {direccion}")
    while True:
        datos = cliente.recv(1024)
        if not datos:
            break
        print(f"Mensaje recibido: {datos.decode()}")
        cliente.sendall(b'Respuesta del servidor')
    cliente.close()
```

Servidor TCP multicliente

```
def servidor_tcp_multihilo(host='localhost',
puerto=12345):
    with socket.socket(socket.AF_INET,
socket.SOCK_STREAM) as servidor:
        servidor.bind((host, puerto))
```

```
        servidor.listen(5)
        print(f"Servidor escuchando en {host}:
{puerto}")
        while True:
            cliente, direccion = servidor.accept()
            hilo =
threading.Thread(target=manejar_cliente, args=
(cliente, direccion))
            hilo.start()

servidor_tcp_multihilo()
```

Nota: Considera usar **multiprocesos** (multiprocessing) en lugar de hilos para tareas más intensivas en CPU, ya que Python tiene un **Global Interpreter Lock (GIL)** que limita la ejecución concurrente de hilos en un solo núcleo.



